



CUADERNO DE PRÁCTICAS

Desarrollo web con Docker, CodeIgniter 4, React, Next.js e IA

Itinerario autónomo de 4 a 8 semanas

Alumno/a

Daniel Aguilar

Tutor/a

Fecha de inicio

28/08/2026

Fecha de finalización

Versión: septiembre de 2026

Índice

Este cuaderno se organiza en un bloque de nivelación inicial, cuatro semanas obligatorias y cuatro semanas de ampliación.

N.º	Bloque
1	Cómo utilizar este cuaderno
2	Forma de trabajo: autonomía con puntos de control
3	Itinerario general y ritmo individual
4	Nivelación: Visual Studio Code y terminal
5	Nivelación: Git
6	Nivelación: Docker - conceptos y primer contenedor
7	Nivelación: Docker Compose
8	Semana 1 - Docker + CodeIgniter 4 + MySQL
9	Semana 2 - API REST con CodeIgniter 4
10	Semana 3 - React y Next.js
11	Semana 4 - Integración y proyecto obligatorio
12	Semana 5 - Autenticación y autorización
13	Semana 6 - Testing, debugging y calidad
14	Semana 7 - Programación apoyada en IA
15	Semana 8 - Reto final autónomo
16	Organización diaria recomendada
17	Entregables y revisiones
18	Criterios de evaluación
19-22	Checklists de salida
23	Reflexión final
24	Referencia rápida de entregables

1. Cómo utilizar este cuaderno

Este cuaderno está diseñado para alumnos en prácticas de desarrollo web que ya cuentan con una base previa de HTML, CSS, JavaScript y PHP, pero que pueden llegar con niveles muy diferentes en Visual Studio Code, Git, Docker, Codelgniter, React o Next.js.

Idea principal: no todos los alumnos deben avanzar a la misma velocidad. Cada bloque tiene un objetivo y una prueba de salida. Quien ya domine una herramienta podrá demostrarlo y avanzar; quien no la conozca dispondrá de ejercicios para aprenderla.

Objetivo de las prácticas

- Trabajar con autonomía en un entorno de desarrollo similar al de una empresa.
- Investigar antes de pedir una solución.
- Construir y desplegar aplicaciones reproducibles con Docker.
- Desarrollar un backend API con Codelgniter 4 y MySQL.
- Desarrollar un frontend con React y Next.js.
- Integrar frontend, backend y base de datos.
- Utilizar Git de forma continua y documentar el trabajo realizado.
- Utilizar inteligencia artificial como herramienta de apoyo sin delegar en ella la comprensión del código.

Duración

El itinerario obligatorio cubre aproximadamente 4 semanas a jornada completa. Las semanas 5 a 8 amplían el proyecto con autenticación, testing, debugging, calidad, uso avanzado de IA y un reto final autónomo.

2. Forma de trabajo: autonomía con puntos de control

El tutor define objetivos, revisa entregables y desbloquea problemas. El alumno es responsable de investigar, probar, validar y documentar su solución.

Ciclo de trabajo para cada tarea

1. Leer el objetivo y los criterios de aceptación.
2. Investigar en documentación oficial y fuentes técnicas fiables.
3. Plantear una solución antes de escribir grandes cantidades de código.
4. Implementar por pasos pequeños y comprobables.
5. Probar casos correctos y casos de error.

6. Revisar el código y eliminar pruebas temporales.
7. Realizar un commit descriptivo.
8. Documentar los problemas importantes y lo aprendido.

Antes de pedir ayuda

Regla de autonomía: antes de consultar al tutor, el alumno debe ser capaz de explicar qué intenta hacer, qué esperaba, qué ocurre realmente, qué error aparece, qué ha probado y qué documentación o consultas ha realizado.

Qué intento hacer:

Qué esperaba que ocurriera:

Qué ocurre realmente:

Error o síntoma:

Qué he probado:

Qué documentación he consultado:

Mi hipótesis actual:

3. Itinerario general y ritmo individual

Bloque	Contenido	Salida esperada	Duración orientativa
Nivelación	VS Code, terminal, Git y Docker	Entorno reproducible y prueba técnica	0,5 - 5 días
Semana 1	Docker + Codelgniter 4 + MySQL	Backend arrancando con Compose	Resto de la semana
Semana 2	API REST y modelo de datos	API CRUD con relaciones y validación	1 semana
Semana 3	React + Next.js	Frontend conectado a la API	1 semana
Semana 4	Integración y proyecto obligatorio	Aplicación full stack dockerizada	1 semana
Semana 5	Autenticación y permisos	Login, roles y rutas protegidas	Opcional / ampliación
Semana 6	Testing, debugging y calidad	Proyecto más robusto y depurado	Opcional / ampliación
Semana 7	Programación apoyada en IA	Refactor y revisión asistida	Opcional / ampliación
Semana 8	Reto final autónomo	Nueva aplicación desde un briefing	Opcional / ampliación

Pruebas de salida

Los bloques de nivelación no se superan por tiempo, sino por capacidad. Un alumno que ya conozca Docker puede completar la prueba de salida directamente. Si la supera, continúa. Si no, realiza los ejercicios correspondientes.

Importante: avanzar rápido no consiste en saltarse conceptos, sino en demostrar que se dominan. Del mismo modo, necesitar más tiempo en el bloque inicial no se considera un problema si el alumno progresa y documenta su aprendizaje.

4. Bloque de nivelación: Visual Studio Code y terminal

Objetivo

Ser capaz de utilizar VS Code y la terminal como herramientas habituales de trabajo sin depender del tutor para operaciones básicas.

Conocimientos mínimos

- Abrir una carpeta de proyecto y navegar por su estructura.
- Crear, mover y renombrar archivos y carpetas.
- Utilizar la terminal integrada.
- Buscar texto en todo el proyecto.
- Trabajar con varias pestañas y dividir el editor.
- Instalar y configurar extensiones.
- Interpretar errores y avisos del editor.
- Formatear código y comprender la función de ESLint y Prettier.

Ejercicio VS-01

Crear desde VS Code la siguiente estructura y documentar los pasos realizados:

```
nivelacion-vscode/  
├─ src/  
│   ├── css/  
│   └─ js/  
├─ docs/  
└─ README.md
```

- Abrir una terminal integrada y crear o modificar al menos un archivo desde ella.
- Buscar una palabra en todo el proyecto.
- Instalar una extensión útil para desarrollo.

- Aplicar formato automático a un archivo.
- Crear una configuración local del proyecto en .vscode cuando sea necesaria.

Preguntas de comprobación

- ¿Qué es una extensión de VS Code?

Una herramienta instalable en VSCode

- ¿Qué función cumplen Prettier y ESLint?

Prettier: una extensión que embellece la presentación del código

ESLint: herramienta de análisis de código para encontrar errores

- ¿Qué ventaja tiene una configuración de proyecto en .vscode frente a una configuración únicamente personal?

Permite darle consistencia a un proyecto en el que participan varios usuarios.

Prueba de salida VS Code

El tutor podrá solicitar una pequeña modificación de estructura, una búsqueda global, el uso de terminal y la instalación/configuración de una extensión. Si el alumno lo resuelve sin instrucciones paso a paso, el bloque se considera superado.

5. Bloque de nivelación: Git

Git será obligatorio durante todas las prácticas. El repositorio debe reflejar una evolución real del proyecto; no se aceptará un único commit final.

Conceptos mínimos

- Repositorio
- Commit
- Branch
- Merge
- Clone
- Pull
- Push
- Remote
- .gitignore
- Conflicto

Ejercicio GIT-01

1. Inicializar un repositorio para el ejercicio anterior.
2. Crear un .gitignore apropiado.
3. Realizar un primer commit.
4. Modificar varios archivos y realizar un segundo commit.
5. Crear una rama feature/README.
6. Realizar un cambio en la rama e integrarlo en la rama principal.
7. Consultar el historial e identificar los commits realizados.

Convención de commits

feat: añade endpoint de clientes
fix: corrige validación de email
refactor: separa servicio de proyectos
docs: actualiza instrucciones de instalación

Prueba de salida Git

El alumno deberá explicar la diferencia entre add, commit y push, crear una rama, realizar un cambio y fusionarlo. También debe saber identificar qué archivos no deberían versionarse.

6. Bloque de nivelación: Docker - conceptos y primer contenedor

Conceptos que debes poder explicar

- Imagen
- Contenedor
- Registro
- Puerto
- Volumen
- Red
- Dockerfile
- Docker Compose

No memorices definiciones: debes poder explicar cada concepto con tus propias palabras y relacionarlo con el proyecto que estás ejecutando.

Ejercicio DOCKER-01: primer contenedor

Ejecutar un contenedor Nginx accesible en <http://localhost:8080>. No se proporciona el comando.

- Descargar la imagen necesaria.
- Crear el contenedor y publicar el puerto.
- Consultar contenedores activos y detenidos.
- Revisar los logs.
- Entrar en el contenedor.
- Detenerlo, iniciarlo de nuevo y eliminarlo.

Preguntas

- ¿Qué significa 8080:80?

8080: es el puerto del host / "máquina física"

80: es el puerto por donde escucha el contenedor

- ¿Dónde está cada puerto?

- ¿Qué ocurre si el puerto 8080 está ocupado?

Al intentar conectarnos por ese puerto recibiremos un error. No será posible establecer conexión con ese contenedor.

- ¿Podrías publicar 9000:80? ¿Qué cambiaría?

Si. En cuanto a la url, habría que utilizar localhost:9000. Aunque se pueda utilizar, está enfocado a admin infraestructura y/o php

Ejercicio DOCKER-02: primera imagen propia

```
primer-dockerfile/  
├─ index.html  
├─ css/  
│   └─ style.css  
└─ Dockerfile
```

Crear una imagen que sirva la web mediante un servidor web. Investigar FROM, WORKDIR, COPY, RUN, EXPOSE, CMD y ENTRYPOINT. No es obligatorio utilizar todas las instrucciones; sí explicar las que se utilicen.

Ejercicio DOCKER-03: volúmenes

Modificar el HTML desde el equipo local sin reconstruir la imagen cada vez. Investigar cómo montar un volumen y explicar cuándo conviene copiar archivos en la imagen y cuándo usar un volumen.

7. Bloque de nivelación: Docker Compose

Objetivo

Levantar varios servicios mediante un único archivo Compose y comprender cómo se comunican entre ellos.

Ejercicio COMPOSE-01

Servicios:

- web
- database

Crear un compose.yml con ambos servicios. La base de datos debe conservar la información aunque el contenedor sea eliminado y creado de nuevo.

- Investigar services, ports, environment, volumes y depends_on.
- Crear un volumen persistente para la base de datos.
- Utilizar variables de entorno.
- Comprobar la red creada por Docker Compose.

```
docker compose up -d --build
```

Pregunta clave

¿localhost? si el backend y la base de datos están en contenedores distintos, investiga por qué el backend normalmente no debe utilizar localhost como host de la base de datos.

Prueba de salida del bloque de nivelación

Crear sin instrucciones paso a paso una pequeña web servida mediante Docker y añadir una base de datos MySQL con Docker Compose, persistencia y README. Si el alumno ya dominaba Docker, puede intentar esta prueba directamente al inicio.

1. Crear una tabla e insertar un dato

```
docker exec -it composer01-db mariadb -u dani -pdani1234 mi_db -e "CREATE TABLE prueba (id INT, nombre VARCHAR(50)); INSERT INTO prueba VALUES (1, 'Persistencia Ok');"
```

2. Destruir los contenedores (esto NO borra el volumen db_data)

```
docker compose down
```

3. Volver a levantar el entorno

```
docker compose up -d
```

4. Verificar que la tabla y el dato siguen intactos

```
docker exec -it composer01-db mariadb -u dani -pdani1234 mi_db -e "SELECT * FROM prueba;"
```

8. Semana 1 - Docker + CodeIgniter 4 + MySQL

Objetivo semanal

Crear un backend CodeIgniter 4 dentro de un entorno Docker reproducible y conectado a MySQL.

Antes de empezar: entender CodeIgniter 4

Antes de comenzar las tareas de la semana, dedica un tiempo a entender cómo organiza CodeIgniter una aplicación. No es necesario memorizar su API: el objetivo es disponer de un mapa mental que te permita saber dónde buscar y por qué cada pieza existe.

¿Qué es CodeIgniter 4?

CodeIgniter 4 es un framework PHP orientado al desarrollo de aplicaciones web. Proporciona una estructura de proyecto, routing, acceso a base de datos, validación, herramientas de línea de comandos y componentes para construir aplicaciones y APIs sin tener que resolver desde cero problemas habituales.

MVC como mapa mental

CodeIgniter favorece una organización basada en MVC. Para estas prácticas puedes pensar inicialmente en el flujo:

```
Petición HTTP
↓
Route
↓
Controller
↓
Model
↓
Base de datos
↓
Response JSON
```

No todas las peticiones tienen que utilizar todas estas piezas, pero esta secuencia te ayudará a localizar responsabilidades y evitar que toda la lógica termine concentrada en un único archivo.

Conceptos básicos que debes conocer

- Routes: relacionan una URL y un método HTTP con el código que debe atender la petición.
- Controllers: reciben la petición, coordinan la operación y construyen la respuesta. Evita convertirlos en archivos con toda la lógica de la aplicación.
- Models: encapsulan el acceso habitual a los datos de una entidad o tabla.



- Migrations: describen mediante código la evolución de la estructura de la base de datos.
- Seeders: permiten generar datos iniciales o de prueba de forma reproducible.
- Validation: comprueba que los datos recibidos cumplen las reglas de la aplicación antes de procesarlos.
- Request y Response: representan la información que entra en la aplicación y la respuesta HTTP que sale de ella.
- .env: permite configurar el proyecto según el entorno sin escribir credenciales y parámetros variables directamente en el código.
- Spark: es la herramienta de línea de comandos de CodeIgniter para ejecutar tareas del framework, como migraciones o comandos de desarrollo.

Carpetas que vas a utilizar

- app/Config/: configuración y rutas.
- app/Controllers/: controladores.
- app/Models/: modelos.
- app/Database/Migrations/: migraciones.
- app/Database/Seeds/: seeders.
- public/: punto de entrada público de la aplicación y recursos públicos.
- writable/: logs, caché, sesiones y otros archivos que la aplicación necesita escribir.
- vendor/: dependencias instaladas mediante Composer. No debe editarse manualmente.

Documentación oficial recomendada

- [Guía de usuario de CodeIgniter 4](#)
- [Tutorial oficial de CodeIgniter 4](#)
- [Routing](#)
- [Controllers](#)
- [Models](#)
- [Migrations](#)
- [RESTful Resource Handling](#)

Utiliza la documentación oficial como referencia principal. Si encuentras una solución en una IA, foro o tutorial, contrástala con la documentación cuando afecte a la arquitectura, configuración, seguridad o comportamiento del framework.

Antes de continuar, debes poder responder

- ¿Qué diferencia existe entre una Route, un Controller y un Model?
 - ROUTE redirige a una URL y qué 'controller' realiza la acción -> redirige tráfico HTTP
 - CONTROLLER: recibe peticiones del user y solicita datos al 'modelo (capa negocio)'
 - MODELO: tras la "petición" del controller interactúa con la BD y obtiene la información
- ¿Por qué conviene crear la base de datos mediante migrations en lugar de modificarla manualmente?



Nos permite tener un control de versiones y trazabilidad sobre la BD. Estos cambios quedan como código PHP en el repo, además, permiten hacer rollbacks

- ¿Qué información colocarías en .env y qué información no deberías subir al repositorio?

En '.env' se incluyen datos que cambian según el entorno (credenciales bd, claves api, configuraciones del server...) Como esta información es sensible, no se debe subir el '.env' al repo. Lo normal sería incluir el archivo en el '.gitignore'

- ¿Qué recorrido aproximado sigue una petición GET /api/clientes hasta devolver JSON?
front hace petición -> el router reconoce el método GET y redirige al controlador -> el controller ejecuta la función para hacer el fetching

Proyecto base

```
proyecto/  
├── backend/  
├── docker/  
├── compose.yml  
├── .env.example  
└── README.md
```

Tareas

1. Crear el repositorio y la estructura inicial.
2. Crear el proyecto CodeIgniter 4 mediante Composer.
3. Investigar la estructura app/, public/, writable/, vendor/ y .env.
4. Crear un Dockerfile adecuado para PHP y CodeIgniter.
5. Crear un servicio MySQL en Docker Compose.
6. Conectar CodeIgniter con la base de datos utilizando variables de entorno.
7. Crear una primera migración y un seeder.
8. Crear una ruta de comprobación /api/health que permita verificar que el backend está funcionando.
9. Documentar cómo arrancar el proyecto desde cero.

Criterios de aceptación

- El backend se ejecuta sin instalar PHP ni MySQL de forma manual para ejecutar el proyecto.
- docker compose up -d levanta los servicios.
- Los datos de MySQL persisten.
- El README permite a otra persona arrancar el proyecto.
- No se suben credenciales reales al repositorio.

Checkpoint del tutor

El tutor puede pedir un cambio no previsto, por ejemplo cambiar el puerto del backend o añadir una nueva variable de entorno. El alumno debe localizar dónde realizar el cambio y explicar su efecto.

9. Semana 2 - API REST con CodeIgniter 4

Dominio del proyecto

A partir de esta semana el proyecto tendrá varias entidades relacionadas. Se propone el siguiente dominio:

CLIENTE 1 — N PROYECTOS 1 — N TAREAS

Entidad	Campos mínimos
Cliente	id, nombre, cif, email, telefono, estado, created_at, updated_at
Proyecto	id, cliente_id, nombre, descripcion, estado, fecha_inicio, fecha_fin
Tarea	id, proyecto_id, titulo, descripcion, prioridad, estado, fecha_limite

Tareas backend

- Diseñar las migraciones y relaciones.
- Crear Models y Controllers.
- Crear seeders con datos de prueba.
- Implementar CRUD REST.
- Validar los datos recibidos.
- Devolver códigos HTTP coherentes.
- Gestionar recursos inexistentes y errores.
- Añadir búsqueda, filtros y paginación al menos en un recurso.
- Probar los endpoints mediante Postman, Insomnia o una herramienta equivalente.

Ejemplos de endpoints

```
GET    /api/clientes
GET    /api/clientes/{id}
POST   /api/clientes
PUT    /api/clientes/{id}
DELETE /api/clientes/{id}
```

```
GET /api/proyectos?cliente=12&estado=activo
GET /api/tareas?proyecto=8&estado=pendiente&page=2
```

Preguntas de comprensión

- ¿Qué diferencia existe entre 400, 401, 403, 404 y 422?

- ¿Qué validaciones deben estar obligatoriamente en backend aunque existan en frontend?

- ¿Qué ocurre si se elimina un cliente que tiene proyectos asociados?

- ¿Dónde debe resolverse la lógica de filtrado: frontend, backend o ambos? Justifica la respuesta.

10. Semana 3 - React y Next.js

Objetivo semanal

Crear el frontend de la aplicación y consumir la API desarrollada durante la semana anterior.

Nivelación rápida de React

Antes de comenzar, comprueba que puedes explicar componente, props, state, evento, hook, useState y useEffect. Si alguno de estos conceptos no está claro, realiza ejercicios breves antes de continuar.

React: conceptos que necesitas para trabajar con Next.js

Next.js utiliza React para construir la interfaz. Por eso, antes de aprender las características propias de Next.js debes entender algunos conceptos de React. No necesitas dominar todo React para comenzar, pero sí saber reconocer estas piezas en el código.

- Componente: función reutilizable que representa una parte de la interfaz y devuelve JSX.
- Props: datos que un componente recibe desde su componente padre. Se utilizan para parametrizar y reutilizar componentes.
- State: información que un componente necesita recordar entre renderizados y que puede cambiar durante la interacción.
- Eventos: acciones como click, submit o change que permiten responder a la interacción del usuario.
- useState: Hook habitual para almacenar estado local en un componente.
- useEffect: Hook para sincronizar un componente con sistemas externos. No debe utilizarse automáticamente para cualquier lógica.
- Renderizado de listas y condicionales: permite construir la interfaz a partir de datos y estados.
- Formularios controlados: campos cuyo valor se gestiona desde el estado de React.

¿Qué aporta Next.js?

Next.js es un framework basado en React que añade estructura, routing, renderizado en servidor, optimizaciones y herramientas para construir aplicaciones web completas. En estas prácticas utilizaremos el App Router.

- app/: directorio principal del App Router.
- page.js / page.tsx: define la interfaz de una ruta.
- layout.js / layout.tsx: define una estructura compartida entre varias páginas.
- Rutas dinámicas: carpetas como [id] permiten representar URLs como /clientes/25.



- **Server Components:** en el App Router, los componentes son de servidor por defecto. Son útiles para renderizar contenido y trabajar con datos sin enviar toda la lógica al navegador.
- **Client Components:** se utilizan cuando necesitas interacción en el navegador, estado, eventos, determinados Hooks o APIs del navegador. Se marcan con la directiva "use client".
- **Data fetching:** los datos pueden cargarse en el servidor o en el cliente según las necesidades de la pantalla.
- **Variables de entorno:** las variables que deben estar disponibles en el navegador requieren especial atención; no se deben exponer secretos al cliente.
- **Services:** aunque no sea una obligación de Next.js, en este proyecto centralizaremos la comunicación con la API de CodeIgniter para evitar llamadas dispersas y duplicadas.

Mapa mental de nuestra aplicación

```
graph TD
  Navegador --> NextJS[Next.js / React]
  NextJS --> HTTP[HTTP + JSON]
  HTTP --> API[API CodeIgniter 4]
  API --> MySQL[MySQL]
```

React se ocupa principalmente de la interfaz y la interacción. Next.js organiza la aplicación React y decide dónde se ejecutan distintas partes. CodeIgniter seguirá siendo el backend responsable de la API y de la lógica de acceso a los datos de este proyecto.

Documentación oficial recomendada

- [React - Learn](#)
- [React - Passing Props to a Component](#)
- [React - State: A Component's Memory](#)
- [React - Built-in Hooks](#)
- [Next.js - Documentation](#)
- [Next.js - App Router](#)
- [Next.js - Server and Client Components](#)
- [Next.js - Fetching Data](#)
- [Next.js - Environment Variables](#)

Antes de continuar, debes poder responder

- ¿Qué diferencia existe entre props y state?

- ¿Qué problema resuelve un componente reutilizable?

- ¿Qué diferencia conceptual existe entre React y Next.js?

- ¿Qué diferencia existe entre un Server Component y un Client Component?

- ¿Cuándo necesitarías utilizar "use client"?

- ¿Por qué una contraseña o una clave privada nunca debería exponerse en una variable enviada al navegador?

- ¿Cómo viajarán los datos desde MySQL hasta una página de clientes en nuestro proyecto?

Fast track: si ya conoces React, no es necesario dedicar una jornada a contadores o ejercicios aislados. Demuestra que entiendes los conceptos y comienza el frontend real.

Estructura recomendada

```
frontend/  
├─ app/  
├─ components/  
├─ services/  
├─ types/  
├─ utils/  
└─ public/
```

Pantallas mínimas

```
/dashboard  
/clientes  
/clientes/nuevo
```

```
/clientes/[id]
/clientes/[id]/editar
/proyectos
/proyectos/[id]
/tareas
```

Requisitos

- Crear componentes reutilizables para elementos repetidos.
- Centralizar las llamadas a API en una capa de servicios.
- Gestionar loading, error y estado vacío.
- Crear formularios con validación de cliente.
- Mostrar los errores devueltos por el backend.
- Utilizar variables de entorno para la URL del backend.
- Evitar duplicación innecesaria de código.

Checkpoint

El tutor puede pedir añadir un nuevo campo a Cliente. El alumno deberá identificar todos los cambios necesarios: migración, modelo, validación, API, formulario, listado y detalle.

11. Semana 4 - Integración y proyecto obligatorio

Arquitectura final

```
Usuario
  ↓
Next.js
  ↓ HTTP / JSON
CodeIgniter 4
  ↓
MySQL
```

Todo el entorno debe poder iniciarse mediante Docker Compose.

Servicios mínimos

- frontend - Next.js
- backend - CodeIgniter 4 / PHP
- database - MySQL o MariaDB

Requisitos funcionales mínimos

- CRUD completo de clientes.
- CRUD completo de proyectos.



- CRUD de tareas o funcionalidad equivalente.
- Relaciones entre entidades.
- Búsqueda y filtros.
- Paginación.
- Validación frontend y backend.
- Gestión de errores.
- Confirmación antes de operaciones destructivas.
- README y .env.example.

Requisito de despliegue

`docker compose up -d`

Una persona que clone el repositorio debe poder levantar el proyecto siguiendo el README, sin tener que reconstruir manualmente carpetas ni copiar archivos a mano.

12. Semana 5 - Autenticación y autorización

Este bloque se realiza si las prácticas continúan más allá de las cuatro semanas o si el alumno avanza con rapidez.

Objetivo

Añadir usuarios, autenticación, rutas protegidas y permisos por rol.

Rol	Permisos mínimos
Administrador	Crear, editar y eliminar clientes, proyectos y tareas. Gestionar usuarios.
Usuario	Consultar información y modificar únicamente las operaciones que determine el ejercicio.

Tareas

- Tabla y modelo de usuarios.
- Hash de contraseñas.
- Login y logout.
- Endpoint de usuario autenticado.
- Protección de rutas del backend.
- Protección de páginas del frontend.
- Diferenciar 401 y 403.
- Evitar almacenar secretos o contraseñas en texto plano.

Reto

Crear una operación que solo pueda ejecutar un administrador y demostrar mediante una petición válida y otra no autorizada que el control se realiza en backend.

13. Semana 6 - Testing, debugging y calidad

Objetivo

Aprender a encontrar fallos de forma sistemática y mejorar un proyecto que ya funciona.

Herramientas de diagnóstico

- docker logs y docker logs -f
- DevTools del navegador: Console y Network
- Logs de CodeIgniter
- Errores de build de Next.js
- Consultas y errores de MySQL
- Pruebas de API

Ejercicios de debugging

El tutor puede entregar una rama deliberadamente rota. El alumno deberá identificar la causa y documentar el proceso de investigación.

- El frontend no conecta con el backend dentro de Docker.
- Los datos desaparecen al recrear el contenedor.
- La API devuelve 500 únicamente al crear un registro.
- Una variable de entorno funciona en desarrollo pero no en producción.
- El frontend realiza dos peticiones innecesarias para la misma información.

Testing

Añadir pruebas automáticas al menos en el backend o en las partes críticas del proyecto. Priorizar validaciones, endpoints y reglas de negocio relevantes.

Calidad de código

- Eliminar duplicación.
- Dividir componentes excesivamente grandes.
- Separar responsabilidades.
- Revisar nombres de funciones y variables.
- Eliminar console.log y código temporal.
- Comprobar que los errores se manejan de forma explícita.

14. Semana 7 - Programación apoyada en inteligencia artificial

Herramienta recomendada

Se podrá utilizar GitHub Copilot en Visual Studio Code u otra herramienta autorizada por el tutor. La herramienta es un apoyo; el alumno sigue siendo responsable de cada línea incorporada al proyecto.

Usos recomendados

- Explicar código existente.
- Investigar causas posibles de un error.
- Proponer casos de prueba.
- Revisar legibilidad y duplicación.
- Sugerir refactorizaciones.
- Generar pequeñas piezas repetitivas después de definir los requisitos.
- Comparar alternativas técnicas.

Usos que deben evitarse

- Pedir que genere el proyecto completo sin comprender la solución.
- Copiar una respuesta sin probarla.
- Incorporar dependencias o código sin comprobar su procedencia y función.
- Compartir contraseñas, claves, datos personales o información confidencial en prompts.

Regla de comprensión

Todo código entra en la revisión: si utilizas IA para crear una función, un Dockerfile, una consulta o un componente, debes poder explicar qué hace, por qué lo utilizas y qué cambiaría si modificases una parte relevante.

Ejercicio IA-01: diagnóstico

Elegir un error real encontrado durante el proyecto y utilizar IA para generar hipótesis, no para pedir la solución completa. Registrar en docs/ia.md el prompt utilizado, las hipótesis propuestas, cuáles se comprobaron y cuál era la causa real.

Ejercicio IA-02: revisión de código

Seleccionar un controller o componente suficientemente grande y pedir a la IA que detecte problemas de legibilidad, duplicación, responsabilidad o posibles bugs sin reescribirlo. Aplicar únicamente las recomendaciones justificadas.

Ejercicio IA-03: tests

Pedir a la IA una lista de casos de prueba para un endpoint. Revisar la lista, eliminar pruebas irrelevantes, añadir las que falten y ejecutar los casos seleccionados.

Ejemplos de prompts adecuados

No modifiques el código todavía. Analiza este error y enumera posibles causas ordenadas por probabilidad. Para cada causa, dime cómo comprobarla.

Revisa este componente buscando duplicación, responsabilidades mezcladas y posibles bugs. No lo reescribas; quiero decidir qué cambios aplicar.

Tengo este endpoint y estas reglas de validación. Propón casos de prueba normales, límite y de error. Indica qué resultado debería esperar en cada uno.

15. Semana 8 - Reto final autónomo

Briefing

Desarrollar una aplicación de gestión de incidencias. A diferencia de los bloques anteriores, no se proporcionará una secuencia de pasos técnicos.

Entidades mínimas

- Usuarios
- Clientes
- Incidencias
- Comentarios

Una incidencia deberá incluir

- Cliente
- Título
- Descripción
- Prioridad
- Estado
- Responsable
- Fecha de creación
- Fecha de resolución cuando corresponda

Requisitos

- Login y roles.
- CRUD y relaciones.
- Buscador y filtros.
- Paginación.
- Dashboard sencillo.
- API REST con CodeIgniter 4.
- Frontend Next.js.
- MySQL.
- Docker Compose.
- README y documentación técnica.

Decisiones que debe tomar el alumno

- Modelo de datos.
- Relaciones.
- Endpoints.
- Estructura del frontend.
- Componentes reutilizables.
- Estrategia de autenticación.
- Variables de entorno.
- Organización de Docker.
- Pruebas prioritarias.

Objetivo del reto: comprobar si el alumno puede convertir un briefing funcional en una solución técnica sin recibir un tutorial paso a paso.

16. Organización diaria recomendada

La jornada no se divide en clases obligatorias. Se recomienda una rutina que facilite el trabajo autónomo y la revisión continua.

Momento	Actividad
Inicio de jornada	Revisar objetivo del día y tareas pendientes.
Primer bloque	Investigación y planificación de la tarea.
Bloque principal	Desarrollo y pruebas.
Antes de finalizar	Revisión, limpieza de código y pruebas.
Cierre	Commit, push y actualización del diario.

Diario de prácticas

Mantener un archivo docs/diario.md o DIARIO.md. No es necesario escribir mucho; debe servir para dejar constancia del proceso de aprendizaje.

14/10/2026

Trabajo realizado

Configuración de Docker Compose para backend y base de datos.

Problema

El backend no podía conectar con MySQL.

Investigación

Revisé logs, configuración y networking de Docker.

Causa

Estaba utilizando localhost como host de la base de datos.

Solución

Utilicé el nombre del servicio definido en Compose.

Aprendido

Dentro de un contenedor, localhost referencia ese mismo contenedor.

17. Entregables y revisiones

Entrega semanal

- Repositorio actualizado.
- Aplicación funcionando.

- README actualizado.
- Diario de prácticas actualizado.
- Problemas importantes documentados.
- Demostración breve del trabajo realizado.

Revisión del tutor

El tutor podrá solicitar pequeños cambios no preparados para comprobar comprensión real. Ejemplos: añadir un campo, cambiar una relación, modificar un puerto, crear un filtro, explicar un fragmento generado con IA o corregir una configuración de Docker.

Documentación mínima del proyecto

- Descripción.
- Tecnologías.
- Requisitos.
- Instalación y arranque.
- Variables de entorno.
- URLs de frontend y backend.
- Base de datos y migraciones.
- Endpoints principales.
- Estructura del proyecto.
- Decisiones técnicas relevantes.

18. Criterios de evaluación

La evaluación debe valorar el resultado, pero también la capacidad para llegar a él de forma autónoma.

Área	Peso	Qué se observa
Autonomía e investigación	20 %	Busca, contrasta, prueba y explica antes de pedir una solución.
Docker y entorno	15 %	Dockerfile, Compose, redes, volúmenes, variables y logs.
Backend	20 %	CodeIgniter, API, base de datos, validaciones y errores.
Frontend	20 %	React/Next.js, componentes, estado, formularios y consumo de API.
Git y proceso	10 %	Commits, ramas, historial y organización.
Calidad y debugging	10 %	Legibilidad, pruebas, diagnóstico y corrección.

Área	Peso	Qué se observa
IA y documentación	5 %	Uso crítico de IA, README y diario.

Qué no se considera autonomía

- Copiar código sin entenderlo.
- Pedir al tutor la solución antes de investigar.
- Pedir a una IA que construya todo el ejercicio y limitarse a pegar el resultado.
- Realizar un único commit al final.
- Ocultar errores o no documentar bloqueos importantes.

19. Checklist de salida - VS Code, Git y Docker

- ☐ Sé utilizar VS Code y su terminal integrada.
- ☐ Sé buscar y navegar por un proyecto.
- ☐ Sé utilizar Git para el trabajo diario.
- ☐ Sé crear y utilizar ramas.
- ☐ Sé explicar qué es una imagen y un contenedor.
- ☐ Sé crear y ejecutar un Dockerfile.
- ☐ Sé consultar logs y entrar en un contenedor.
- ☐ Entiendo puertos, redes y volúmenes.
- ☐ Sé crear un entorno con Docker Compose.
- ☐ Sé utilizar variables de entorno sin publicar secretos.

20. Checklist de salida - Backend

- ☐ Entiendo la estructura de CodeIgniter 4.
- ☐ Sé crear rutas, Controllers y Models.
- ☐ Sé crear y ejecutar migraciones y seeders.
- ☐ Sé diseñar relaciones sencillas de base de datos.
- ☐ Sé crear una API REST.
- ☐ Sé validar datos en backend.
- ☐ Sé devolver códigos HTTP coherentes.
- ☐ Sé implementar búsqueda, filtros o paginación.
- ☐ Sé diagnosticar errores utilizando logs.

21. Checklist de salida - Frontend

- ☐ Entiendo componentes, props y estado.
- ☐ Sé crear rutas y layouts en Next.js.
- ☐ Sé crear formularios y gestionar eventos.
- ☐ Sé consumir una API.
- ☐ Sé gestionar loading, error y estado vacío.
- ☐ Sé separar llamadas a API de los componentes cuando corresponde.
- ☐ Sé reutilizar componentes.
- ☐ Sé utilizar variables de entorno.

22. Checklist de salida - Autonomía e IA

- ☐ Antes de pedir ayuda explico qué he probado.
- ☐ Sé utilizar documentación oficial.
- ☐ Puedo explicar el código que he escrito o generado con IA.
- ☐ Verifico las sugerencias de una IA antes de aplicarlas.
- ☐ No comparto secretos ni datos confidenciales en prompts.
- ☐ Documento problemas importantes y su solución.
- ☐ Puedo recibir un cambio de requisitos y localizar qué partes del proyecto debo modificar.

23. Reflexión final

1. ¿Qué tecnología te ha resultado más sencilla y por qué?

2. ¿Qué tecnología te ha resultado más complicada y por qué?

3. ¿Cuál ha sido el problema más difícil que has solucionado?

4. ¿Qué pasos seguiste para encontrar la solución?

5. ¿En qué tarea te resultó más útil la IA? ¿Qué tuviste que corregir o comprobar?

6. ¿Qué parte del proyecto mejorarías si dispusieras de más tiempo?

7. Si empezaras de nuevo, ¿qué organizarías de forma diferente?

8. Explica con tus palabras cómo se comunican Next.js, CodeIgniter, MySQL y Docker en tu proyecto.

24. Referencia rápida de entregables

Elemento	Obligatorio 4 semanas	Ampliación 5-8 semanas
Repositorio Git	Sí	Sí
Docker Compose	Sí	Sí
Backend CodeIgniter 4	Sí	Sí
MySQL/MariaDB	Sí	Sí
API REST	Sí	Sí
Frontend Next.js	Sí	Sí
CRUD + relaciones	Sí	Sí
README + .env.example	Sí	Sí
Diario / problemas	Sí	Sí
Autenticación y roles	Opcional si hay tiempo	Sí
Tests	Recomendado	Sí
Registro de uso de IA	Recomendado	Sí
Reto final autónomo	No	Sí

FIN DEL CUADERNO DE PRÁCTICAS

El objetivo no es memorizar herramientas: es aprender a investigar, construir, comprobar y explicar una solución.